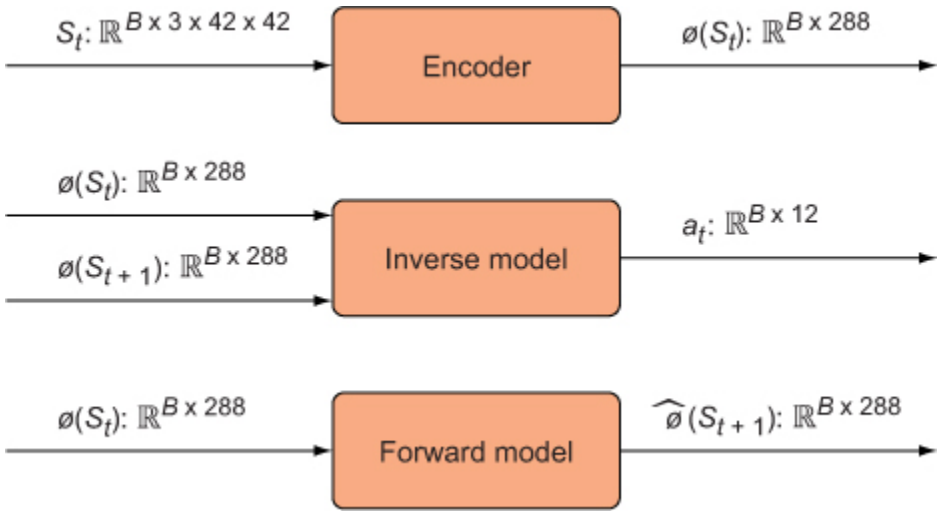
but we do not use an activation function after the output layer. The output layer produces a $B \times 288$ tensor.

Figure 8.14. This figure shows the type and dimensionality of the inputs and outputs of each component of the ICM.



The inverse model is also a simple two-layer neural network with linear layers. The input is two encoded states, S_t and S_{t+1}, concatenated to form a tensor of dimension batch × 288 + 288 = batch × 576. We use a ReLU activation function after the first layer. The output layer produces a tensor of dimension batch × 12 with a softmax function applied, resulting in a discrete probability distribution over actions. When we train the inverse model, we compute the error between this discrete distribution over actions and a one-hot encoded vector of the true action taken.

The encoder is a neural network composed of four convolutional layers (with an identical architecture to the DQN), with an ELU activation function after each layer. The final output is then flattened to get a flat 288-dimensional vector output.

The whole point of the ICM is to produce a single quantity, the forward-model prediction error (figure 8.15). We literally take the error produced by the loss function and use that as the intrinsic reward signal for our DQN. We can add this intrinsic reward to the extrinsic reward to get the final